Pick 1 or 2 Numbers

Two players play on a row of numbers. **Both players play optimally**: each player always chooses moves that maximize their own final result, assuming the other player also plays optimally.

Game Rules

There is an array of **n** integers. Player 1 moves first. On each turn, the current player takes either the first **1** remaining number or the first **2** remaining numbers from the array. The taken numbers are deleted, and their sum is added to that player's score. If only one number is left, the player can take only that one number. The game ends when the array becomes empty.

If Player 1 has a strictly larger score, print **Player 1**. If Player 2 has a strictly larger score, print **Player 2**. Otherwise, print **Draw**.

Input Format

The first line contains an integer **n**. The second line contains **n** space-separated integers **a1, a2, ..., an**. Constraints for the given tests: **1 <= n <= 100**, and values may be negative, zero, or positive.

Sample Input	Sample Output
5 4 -10 7 3 -2	Player 1

Task

Complete the C++ function below. It must return exactly one string: **"Player 1"**, **"Player 2"**, or **"Draw"**.

```
string solve(int n, vector<int> a) {
    // write your code here
}
```

Autograder

A C++ autograder file is provided. It contains an empty **solve(n, a)** function. **Fill only this function.**

```
compile: g++  -O2 autograder.cpp -o autograder
run: ./autograder
```

The autograder reads files named **input0.txt, input1.txt, ...** from the **testcases** folder and compares your answers with matching **output0.txt, output1.txt, ...**. It prints **OK** if all tests pass. Otherwise, it prints the failed testcase number, input, expected output, and your output.

Optional Optimization

A brute force recursive solution is easy to write: try taking 1 number and try taking 2 numbers. This may take exponential time on large inputs. If you think carefully, you can get a polynomial time solution.